

Документация по архитектуре набора инструкций M68k (ISA)

ISA M68k — это простой CISC-подобный набор инструкций, вдохновлённый архитектурой Motorola 68000. В этой документации приводится обзор инструкций, доступных в M68k ISA, их синтаксис и семантика.

Обзор архитектуры

M68k — это 32-битная CISC-архитектура (Complex Instruction Set Computer), вдохновлённая серией Motorola 68000. Она включает:

- 8 регистров данных общего назначения (D0-D7);
- 8 адресных регистров (A0-A7, где A7 используется как указатель стека);
- разнообразные режимы адресации для гибкого доступа к памяти;
- богатый набор инструкций с разными размерами операндов;
- регистр состояния с флагами условий;
- ввод-вывод с отображением в память (Memory-Mapped I/O).

Эта архитектура сочетает простоту RISC-подходов и функциональную насыщенность CISC-процессоров, поэтому отлично подходит для изучения более сложных процессорных решений.

Комментарии в ассемблерном коде M68k обозначаются символом `;`.

- [68000 INSTRUCTION SET](#)
- [Wiki: Motorola 68000](#)

Специфичные представления состояния ISA

- `D<n>:dec`, `D<n>:hex` — регистры данных (D0-D7).
- `A<n>:dec`, `A<n>:hex` — адресные регистры (A0-A7).
- `PC:dec`, `PC:hex` — счётчик команд.
- `SR:bin` — регистр состояния (флаги: N, Z, V, C).

Использование памяти и регистров

- **Регистры данных (D0-D7):** регистры общего назначения для арифметики, логики и манипуляций данными.
- **Адресные регистры (A0-A7):** используются для адресации памяти. A7 также является указателем стека (SP).
- **Регистр состояния (SR):** содержит коды условий (флаги):
 - **N (Negative):** устанавливается, если результат операции отрицательный.
 - **Z (Zero):** устанавливается, если результат операции равен нулю.
 - **V (Overflow):** устанавливается при арифметическом переполнении.
 - **C (Carry):** устанавливается при переносе или заёме в арифметических операциях.

Инструкции

Размер инструкций зависит от режима операции и используемых режимов адресации. Расчёт `ByteSize` учитывает разницу между байтовыми и long-операциями: байтовые операции обычно занимают меньше памяти, чем long.

Режимы операций

Инструкции M68k могут работать с данными разных размеров:

- **Byte (.b):** 8-битные операции
- **Long (.l):** 32-битные операции (по умолчанию)

Пример: `add.b D0, D1` выполняет операцию над младшими байтами D0 и D1.

Примечание по реализации `ByteSize`: Реализация `ByteSize` теперь различает байтовые и long-операции. Для `.b` используется 1 байт данных, для `.l` — 4 байта. Все инструкции переходов (jump/branch) всегда используют long-кодировку размером 6 байт вне зависимости от смещения.

Режимы адресации

M68k ISA поддерживает следующие режимы адресации:

1. Прямая регистровая адресация

- **Data Register Direct:** `D0 ... D7`
- **Address Register Direct:** `A0 ... A7`

2. Непосредственное значение (Immediate): `value`

- Пример: `move.l 42, D0`

3. Адресация памяти

- **Косвенная через адресный регистр: `(An)`**
 - Доступ к памяти по адресу, содержащемуся в `An`
 - Пример: `move.l (A0), D0`
- **Косвенная через адресный регистр с постинкрементом: `(An)+`**
 - Использует адрес в `An`, затем увеличивает `An` на размер операции (1 для byte, 4 для long)
 - Пример: `move.l (A0)+, D0`
- **Косвенная через адресный регистр с преддекрементом: `-(An)`**
 - Уменьшает `An` на размер операции, затем использует получившийся адрес
 - Пример: `move.l -(A0), D0`
- **Косвенная через адресный регистр со смещением: `d(An)`**
 - Использует адрес `An` плюс знаковое 16-битное смещение `d`
 - Пример: `move.l 8(A0), D0` или `move.l -4(A0), D0`

- **Косвенная через адресный регистр с индексом:** `d(An,Xn)`
 - Использует адрес `An` плюс значение `Xn` (регистр данных или адресный регистр) плюс знаковое смещение `d`
 - Пример: `move.l 4(A0,D1), D0` или `move.l 8(A0,A1), D0`

Инструкции перемещения данных

• Move

- **Синтаксис:** `move.<size> <source>, <destination>`
- **Описание:** Переместить данные из источника в назначение.
- **Операция:** `<destination> <- <source>`
- **Примеры:**

```
move.l D0, D1      ; Копировать D0 в D1 (32-бит)
move.b D0, D1      ; Копировать младший байт D0 в младший байт D1
move.l 42, D0       ; Загрузить непосредственное значение 42 в D0
move.l (A0), D0     ; Загрузить значение из памяти по адресу в A0 в D0
move.l D0, (A1)     ; Сохранить значение D0 в память по адресу в A1
move.l (A0)+, D0    ; Загрузить значение из A0, затем увеличить A0 на 4
move.l -(A0), D0    ; Уменьшить A0 на 4, затем загрузить значение из A0
move.l 8(A0), D0    ; Загрузить значение из A0+8
move.l -4(A0), D0   ; Загрузить значение из A0-4
move.l 8(A0,D1), D0 ; Загрузить значение из A0+D1+8
move.l 4(A0,A1), D0 ; Загрузить значение из A0+A1+4
```

• Move to Address Register

- **Синтаксис:** `movea.l <source>, <destination>`
- **Описание:** Переместить данные из источника в адресный регистр назначения.
- **Операция:** `<destination> <- <source>`
- **Примеры:**

```
movea.l 1000, A0    ; Загрузить непосредственное значение 1000 в A0
movea.l D0, A1      ; Копировать D0 в A1
```

Логические инструкции

• Not

- **Синтаксис:** `not.<size> <destination>`
- **Описание:** Выполнить побитовое НЕ над операндом назначения.

- **Операция:** `<destination> <- ~<destination>`

- **Примеры:**

```
not.l D0          ; D0 <- ~D0 (32-бит)
not.b D0          ; Применить NOT к младшему байту D0
```

- **And**

- **Синтаксис:** `and.<size> <source>, <destination>`
- **Описание:** Выполнить побитовое И между источником и назначением.
- **Операция:** `<destination> <- <destination> & <source>`
- **Примеры:**

```
and.l D0, D1      ; D1 <- D1 & D0 (32-бит)
and.b D0, D1      ; AND младших байтов D0 и D1
```

- **Or**

- **Синтаксис:** `or.<size> <source>, <destination>`
- **Описание:** Выполнить побитовое ИЛИ между источником и назначением.
- **Операция:** `<destination> <- <destination> | <source>`
- **Примеры:**

```
or.l D0, D1       ; D1 <- D1 | D0 (32-бит)
or.b D0, D1       ; OR младших байтов D0 и D1
```

- **Exclusive Or**

- **Синтаксис:** `xor.<size> <source>, <destination>`
- **Описание:** Выполнить побитовое исключающее ИЛИ между источником и назначением.
- **Операция:** `<destination> <- <destination> ^ <source>`
- **Примеры:**

```
xor.l D0, D1      ; D1 <- D1 ^ D0 (32-бит)
xor.b D0, D1      ; XOR младших байтов D0 и D1
```

Арифметические инструкции

- **Add**

- **Синтаксис:** `add.<size> <source>, <destination>`
- **Описание:** Прибавить источник к назначению.
- **Операция:** `<destination> <- <destination> + <source>`
- **Примеры:**

```
add.l D0, D1      ; D1 <- D1 + D0 (32-бит)
add.b D0, D1      ; Сложение младших байтов D0 и D1
add.l 10, D0       ; D0 <- D0 + 10
add.b 5, D0        ; Прибавить 5 к младшему байту D0
```

- **Subtract**

- **Синтаксис:** `sub.<size> <source>, <destination>`
- **Описание:** Вычесть источник из назначения.
- **Операция:** `<destination> <- <destination> - <source>`
- **Примеры:**

```
sub.l D0, D1      ; D1 <- D1 - D0 (32-бит)
sub.b D0, D1      ; Вычесть младший байт D0 из младшего байта D1
sub.l 1, D0        ; D0 <- D0 - 1
sub.b 1, D0        ; Вычесть 1 из младшего байта D0
```

- **Compare**

- **Синтаксис:** `cmp.<size> <source>, <destination>`
- **Описание:** Сравнить назначение с источником, вычитая источник из назначения и устанавливая флаги условий, без сохранения результата.
- **Операция:** `temp <- <destination> - <source>` (устанавливает только флаги)
- **Примеры:**

```
cmp.l D0, D1      ; Сравнить D1 с D0 (32-бит)
cmp.b D0, D1      ; Сравнить младшие байты D1 и D0
cmp.l 10, D0       ; Сравнить D0 с непосредственным значением 10
cmp.b 5, D0        ; Сравнить младший байт D0 с 5
cmp.l (A0), D0     ; Сравнить D0 со значением в памяти по адресу A0
```

- **Multiply**

- **Синтаксис:** `mul.<size> <source>, <destination>`
- **Описание:** Умножить назначение на источник.
- **Операция:** `<destination> <- <destination> * <source>`
- **Примеры:**

```
mul.l D0, D1      ; D1 <- D1 * D0 (32-бит)
mul.b D0, D1      ; Умножение младших байтов D0 и D1
```

- **Divide**

- **Синтаксис:** `div.<size> <source>, <destination>`
- **Описание:** Разделить назначение на источник.
- **Операция:** `<destination> <- <destination> / <source>`
- **Примеры:**

```
div.l D0, D1      ; D1 <- D1 / D0 (32-бит)
div.b D0, D1      ; Деление с использованием младших байтов регистров
```

Инструкции сдвигов и вращений

- **Arithmetic Shift Left**

- **Синтаксис:** `asl.<size> <source>, <destination>`
- **Описание:** Арифметически сдвинуть назначение влево на количество бит из источника.
- **Операция:** `<destination> <- <destination> << <source>`
- **Примеры:**

```
asl.l 2, D0       ; D0 <- D0 << 2 (32-бит)
asl.b 2, D0       ; Сдвиг младшего байта D0 влево на 2 бита
asl.l D1, D0      ; D0 <- D0 << D1
```

- **Arithmetic Shift Right**

- **Синтаксис:** `asr.<size> <source>, <destination>`
- **Описание:** Арифметически сдвинуть назначение вправо на количество бит из источника.

- **Операция:** `<destination> <- <destination> >> <source>` (с сохранением знака)

- **Примеры:**

```
asr.l 2, D0      ; D0 <- D0 >> 2 (32-бит)
asr.b 2, D0      ; Сдвиг младшего байта D0 вправо на 2 бита
asr.l D1, D0     ; D0 <- D0 >> D1
```

- **Logical Shift Left**

- **Синтаксис:** `lsl.<size> <source>, <destination>`
- **Описание:** Логически сдвинуть назначение влево на количество бит из источника.
- **Операция:** `<destination> <- <destination> << <source>`
- **Примеры:**

```
lsl.l 2, D0      ; D0 <- D0 << 2 (32-бит)
lsl.b 2, D0      ; Сдвиг младшего байта D0 влево на 2 бита
lsl.l D1, D0     ; D0 <- D0 << D1
```

- **Logical Shift Right**

- **Синтаксис:** `lsr.<size> <source>, <destination>`
- **Описание:** Логически сдвинуть назначение вправо на количество бит из источника.
- **Операция:** `<destination> <- <destination> >> <source>` (с заполнением нулями)
- **Примеры:**

```
lsr.l 2, D0      ; D0 <- D0 >> 2 (32-бит)
lsr.b 2, D0      ; Сдвиг младшего байта D0 вправо на 2 бита
lsr.l D1, D0     ; D0 <- D0 >> D1
```

Инструкции управления потоком

- **Jump**

- **Синтаксис:** `jmp <label>`
- **Описание:** Перейти к указанной метке.
- **Операция:** `PC <- <label>`
- **Примеры:**

```
jmp loop          ; Переход к метке "loop"
```

- **Branch if Carry Clear**

- **Синтаксис:** `bcc <label>`
- **Описание:** Переход к указанной метке, если флаг переноса сброшен.
- **Операция:** `if C == 0 then PC <- <label>`
- **Примеры:**

```
bcc loop          ; Переход к "loop", если флаг C сброшен
```

- **Branch if Carry Set**

- **Синтаксис:** `bcs <label>`
- **Описание:** Переход к указанной метке, если флаг переноса установлен.
- **Операция:** `if C == 1 then PC <- <label>`
- **Примеры:**

```
bcs error         ; Переход к "error", если флаг C установлен
```

- **Branch if Equal**

- **Синтаксис:** `beq <label>`
- **Описание:** Переход к указанной метке, если установлен флаг нуля.
- **Операция:** `if Z == 1 then PC <- <label>`
- **Примеры:**

```
beq done          ; Переход к "done", если флаг Z установлен
```

- **Branch if Not Equal**

- **Синтаксис:** `bne <label>`
- **Описание:** Переход к указанной метке, если флаг нуля сброшен.
- **Операция:** `if Z == 0 then PC <- <label>`
- **Примеры:**


```
bne loop          ; Переход к "loop", если флаг Z сброшен
```

- **Branch if Less Than**

- **Синтаксис:** `blt <label>`
- **Описание:** Переход к указанной метке, если $N \neq V$.
- **Операция:** `if N != V then PC <- <label>`
- **Примеры:**

```
blt less          ; Переход к "less", если результат меньше
```

- **Branch if Greater Than**

- **Синтаксис:** `bgt <label>`
- **Описание:** Переход к указанной метке, если $Z = 0$ и $N = V$.
- **Операция:** `if Z == 0 and N == V then PC <- <label>`
- **Примеры:**

```
bgt greater       ; Переход к "greater", если результат больше
```

- **Branch if Less Than or Equal**

- **Синтаксис:** `ble <label>`
- **Описание:** Переход к указанной метке, если $Z = 1$ или $N \neq V$.
- **Операция:** `if Z == 1 or N != V then PC <- <label>`
- **Примеры:**

```
ble less_or_equal  ; Переход к "less_or_equal", если результат меньше  
или равен
```

- **Branch if Greater Than or Equal**

- **Синтаксис:** `bge <label>`
- **Описание:** Переход к указанной метке, если $N = V$.
- **Операция:** `if N == V then PC <- <label>`

- **Примеры:**

```
bge greater_or_equal ; Переход к "greater_or_equal", если результат  
больше или равен
```

- **Branch if Minus**

- **Синтаксис:** `bmi <label>`

- **Описание:** Переход к указанной метке, если установлен флаг отрицательного результата.

- **Операция:** `if N == 1 then PC <- <label>`

- **Примеры:**

```
bmi negative ; Переход к "negative", если результат  
отрицательный
```

- **Branch if Plus**

- **Синтаксис:** `bpl <label>`

- **Описание:** Переход к указанной метке, если флаг отрицательного результата сброшен.

- **Операция:** `if N == 0 then PC <- <label>`

- **Примеры:**

```
bpl positive ; Переход к "positive", если результат  
положительный или ноль
```

- **Branch if Overflow Clear**

- **Синтаксис:** `bvc <label>`

- **Описание:** Переход к указанной метке, если флаг переполнения сброшен.

- **Операция:** `if V == 0 then PC <- <label>`

- **Примеры:**

```
bvc no_overflow ; Переход к "no_overflow", если флаг V сброшен
```

- **Branch if Overflow Set**

- **Синтаксис:** `bvs <label>`

- **Описание:** Переход к указанной метке, если флаг переполнения установлен.
- **Операция:** `if V == 1 then PC <- <label>`
- **Примеры:**

```
bvs overflow      ; Переход к "overflow", если флаг V установлен
```

- **Jump to Subroutine**

- **Синтаксис:** `jsr <label>`
- **Описание:** Переход к подпрограмме с сохранением адреса возврата в стеке.
- **Операция:** Поместить `PC` в стек, затем `PC <- target address`
- **Примеры:**

```
jsr my_function   ; Вызов подпрограммы my_function
```

- **Return from Subroutine**

- **Синтаксис:** `rts`
- **Описание:** Возврат из подпрограммы: извлечь адрес возврата из стека и перейти по нему.
- **Операция:** Извлечь `PC` из стека
- **Примеры:**

```
rts               ; Возврат из подпрограммы
```

- **Link**

- **Синтаксис:** `link <address_register>, <offset>`
- **Описание:** Создаёт стековый кадр подпрограммы: устанавливает новый указатель кадра и выделяет место в стеке под локальные переменные. Обычно используется в начале функции.
- **Подробная операция:**
 1. Поместить текущее значение указанного адресного регистра (обычно A6, указатель кадра) в стек
 2. Скопировать текущее значение указателя стека (A7) в указанный адресный регистр, тем самым установить новый указатель кадра

3. Добавить смещение к указателю стека (A7), чтобы зарезервировать место под локальные переменные

- Смещение должно быть отрицательным для выделения памяти (например, `-4` выделяет 4 байта)
- Локальные переменные затем доступны через отрицательные смещения от указателя кадра

◦ **Контекст использования:**

- Обычно используется с `A6` как с традиционным регистром указателя кадра
- Локальные переменные адресуются через отрицательные смещения (например, `-4(A6)`)
- Параметры, переданные через стек, доступны по положительным смещениям (например, `8(A6)`)

◦ **Примеры:**

```
link A6, -8      ; Сохранить старый A6 в стеке, установить A6 = SP,
                 вычесть 8 из SP для выделения памяти
move.l D0, -4(A6) ; Сохранить D0 в первой локальной переменной (4
                 байта от указателя кадра)
move.l D1, -8(A6) ; Сохранить D1 во второй локальной переменной (8
                 байт от указателя кадра)
```

• **Unlink**

◦ **Синтаксис:** `unlk <address_register>`

- **Описание:** Освобождает стековый кадр в конце подпрограммы, восстанавливая значения указателя стека и указателя кадра до состояния перед соответствующей `link`.

◦ **Подробная операция:**

1. Скопировать указатель кадра (указанный адресный регистр, обычно `A6`) в указатель стека (`A7`)
 - Это фактически удаляет все локальные переменные, возвращая указатель стека к позиции до выделения памяти
2. Извлечь значение с вершины стека в указанный адресный регистр
 - Это восстанавливает предыдущее значение указателя кадра, сохранённое инструкцией `link`

◦ **Контекст использования:**

- Всегда используется в паре с соответствующей инструкцией `link`
- Обычно затем идёт `rts` (возврат из подпрограммы)
- Корректно освобождает все локальные переменные, созданные в функции

◦ **Примеры:**

```
unlk A6          ; Установить SP = A6 (освобождение локальных), затем  
извлечь сохранённый указатель кадра в A6  
rts              ; Возврат из подпрограммы
```

Полный пример функции:

```
my_function:  
    link A6, -12      ; Создать стековый кадр с 12 байтами под  
    локальные переменные  
    move.l 8(A6), D0   ; Получить параметр, переданный через стек  
    (положительное смещение)  
    move.l D0, -4(A6)  ; Сохранить в локальную переменную  
    (отрицательное смещение)  
    ; ... тело функции ...  
    unlk A6           ; Уничтожить стековый кадр  
    rts               ; Возврат из подпрограммы
```

- **Halt**

- **Синтаксис:** `halt`
- **Описание:** Остановить машину.
- **Операция:** Останов выполнения
- **Примеры:**

```
halt              ; Остановить программу
```